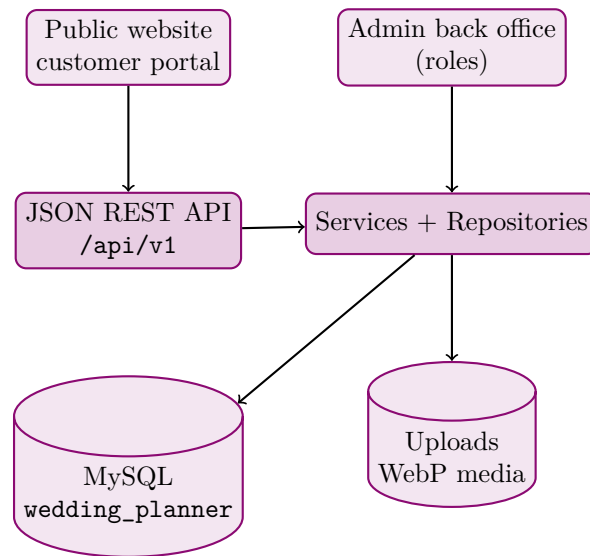


VivaahFlow — Wedding Services Management System Project Documentation

VivaahFlow Engineering

September 2026



Contents

1	Project Title and Identity	2
2	Overview	2
3	Key Features — Public Website	2
4	Key Features — Customer Portal	2
5	Key Features — Admin Panel	2
6	Tech Stack	3
7	System Requirements	3
8	Folder Structure	3
9	Backend Architecture	3
10	Frontend Architecture	4

11 Design System and Theming	5
12 CDN and Third-Party Libraries	5
13 Database Overview	5
14 Database Tables Reference	5
15 Entity Relationships	6
16 Installation (XAMPP)	6
17 Configuration Reference	6
18 Database Setup and Seeding	7
19 Default Credentials	7
20 Running with Apache	7
21 Running with the PHP Built-in Server	7
22 Page and URL Map — Public Website	7
23 Page and URL Map — Portal and Admin	7
24 Authentication Flow	8
25 Session Management	8
26 CSRF Protection	8
27 Security and Rate Limiting	8
28 Roles Overview	9
29 Permissions Matrix	9
30 Full Permission Reference	9
31 Admin Panel Overview	9
32 Dashboard Module	10
33 Customers Module	10
34 Enquiries and Leads Modules	10
35 Catalogue Modules	10

36 Quotations Module	10
37 Bookings Module	10
38 Events and Staff Assignments Module	11
39 Staff Module	11
40 Payments and Invoices Modules	11
41 Reviews Module	11
42 Reports and Analytics Module	11
43 Notifications, Settings, Users, Activity, Profile Modules	11
44 Public Website	11
45 Customer Portal	12
46 Booking and Status Lifecycles	13
47 Calculation Engine and Money Handling	13
48 File Uploads and Media Handling	14
49 Media Integration Pass (September 2026)	14
50 REST API Reference	14
51 Validation and QA Evidence	16
52 Real-Browser Repair Pass (September 2026)	16
53 Troubleshooting, Error Handling and Deployment	17
54 Release Checklist	17

1 Project Title and Identity

VivaahFlow is a wedding services management system: a public marketing website, an authenticated customer self-service portal, and a role-based administrator back office sharing one PHP 8 + MySQL backend behind a single JSON REST API. The visible product name is **Vivaah-Flow**. The database name (`wedding_planner`) and the project folder name are intentionally retained for compatibility. The brand colour is `#8B0A72` on a white theme; the default currency is INR and the timezone is `Asia/Kolkata`.

2 Overview

VivaahFlow manages the full commercial journey of a wedding planning business — from a public website enquiry, through lead follow-up and quotation, into a confirmed booking with scheduled events, assigned staff, recorded payments, generated invoices, and moderated customer reviews — with reporting, notifications, activity auditing, and a role-based permission model. Three surfaces share one backend: the public website (`/`, `/services`, `/packages`, `/gallery`, `/offers`, `/reviews`, `/contact`, `/login`, `/register`), the customer portal (`/account...`), and the admin panel (`/manage...`). All dynamic data is served exclusively through `/api/v1/index.php`; the PHP page files are thin presentation shells containing no SQL.

3 Key Features — Public Website

Branded hero with live statistics; service categories; featured services; a featured package; a gallery showcase; one active offer; testimonials. Service catalogue with keyword search and category filtering; per-service detail pages with gallery, reviews, and similar services. Package detail pages with conditional discount rows and per-service inclusions. An enquiry form capturing contact details, event metadata, and selected services/packages, creating a tracked enquiry with a human-readable reference number.

4 Key Features — Customer Portal

Secure login/registration; the portal is fully gated and redirects anonymous visitors to sign-in with a **next** return path. Dashboard with derived statistics (enquiries, open quotations, active bookings, total paid) and upcoming events. Quotations list and detail with Accept/Decline actions and expiry awareness. Bookings list and detail with payment summaries, event schedules, and assigned staff. Payment history with running totals; invoice viewing/printing; review submission for completed bookings with moderation status and business replies.

5 Key Features — Admin Panel

Role-aware navigation (gated server-side and via **data-permission**). Full CRUD across customers, categories, services, packages, offers, quotations, bookings, events, staff, payments, invoices, and users. Enquiry-to-quotation-to-booking conversion; event scheduling and staff assignment; payment recording with over-payment protection and reversal; invoice lifecycle with printable view; review moderation; reports and analytics (ApexCharts); notifications inbox; business settings with logo upload; activity audit log; per-user profile management.

6 Tech Stack

Layer	Technology
Runtime	PHP 8.2 (strict types, typed properties, constructor promotion)
Database	MySQL/MariaDB 10.4+, PDO with prepared statements
Web server	Apache (XAMPP) or the PHP built-in server
API	Custom JSON router, <code>PATH_INFO</code> routing with <code>?route=</code> fallback
Front end	HTML5, Tailwind CSS (CDN), Flowbite 2.5.1, vanilla JavaScript
Charts	ApexCharts
Icons	Lucide
Fonts	Inter (UI) + Playfair Display (headings)
Date pickers	Flatpickr (lazy library)
Carousels	Swiper 11 (lazy library)

7 System Requirements

PHP 8.2+ with `pdo_mysql`, `mbstring`, `fileinfo`, `session`, `json`, `bcmath` (recommended), `curl` (optional); `gd` is not required. MySQL 5.7+/MariaDB 10.4+. Apache with `mod_rewrite` (recommended) or the PHP built-in server. A modern browser. Internet access for CDN-hosted CSS/JS/fonts/icons (or vendor them locally for offline use).

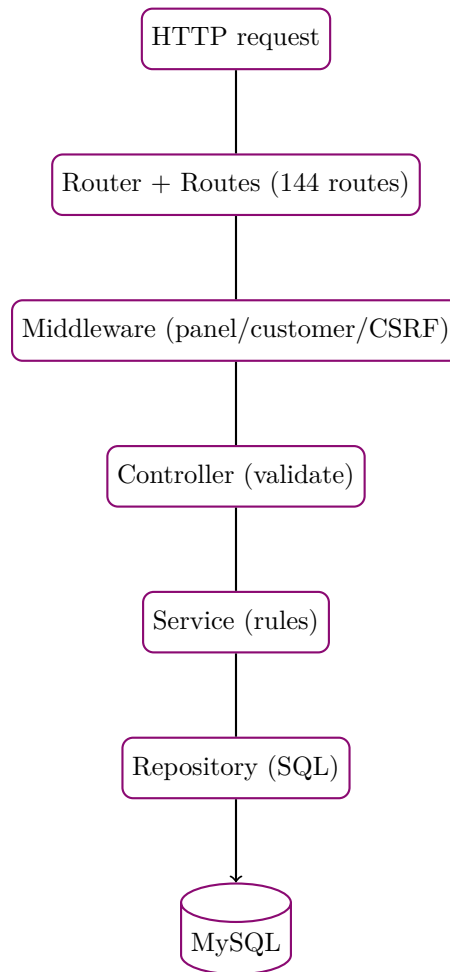
8 Folder Structure

```
VivaahFlow/
  frontend/customer/      public website shells (no SQL) + router.php
  frontend/portal/        authenticated customer area (no SQL) + router.php
  frontend/admin/         admin panel shells (no SQL) + router.php
  frontend/assets/        css/ js/{admin,customer,portal,core}/
                           images/brand/logo.png, icons/
  api/v1/index.php        single JSON API front controller
  backend/                bootstrap, Calculations, Controllers, Core,
                           Database, Helpers, Middleware,
                           Repositories, Services, Validators
  config/config.php       application configuration array
  database/               schema.sql (29 tables), seed.php
  storage/logs/           php-error.log
  uploads/services/       23 WebP service images
  uploads/packages/       3 WebP package covers
  docs/                   this documentation + screenshots + inventory
  index.php, .htaccess, README.md
```

9 Backend Architecture

The backend is layered with dependency injection. The front controller (`api/v1/index.php`) loads `backend/bootstrap.php` and delegates to `App\Core\Router`. `App\` maps to `backend/` (PascalCase, one class per file). `App\Core\Container` autowires constructor dependencies via reflection. `App\Core\Routes::register()` declares all 144 routes with optional middleware; literal sub-paths are registered before `{id}` placeholders. Middleware (`EnsurePanelUser`, `EnsureCustomer`,

`CsrfGuard`) runs before controller actions. Controllers extend `BaseController`, validate via `Validator`, and return standardized JSON. Services enforce business rules and throw `BusinessException`; repositories encapsulate all SQL via PDO prepared statements; `CalculationEngine` centralises money math.



Response envelope: `{success, message, data}`; validation failures return `{success:false, message, errors}` with HTTP 422.

10 Frontend Architecture

Every page loads a shared JS foundation (`AssetManager::SHARED_JS`) then the page module. `app.config.js` bootstraps `window.APP` (base/API URLs, money/date/format helpers, CSRF token). `app.api.js` provides `API.get/post/put/patch/del/upload` resolving with `payload.data` and normalized `ApiError`. `core/loader.js` is the single loading system (progress bar auto-tracked per API request, skeletons, empty states). `app.alerts.js` toasts + promise confirm; `app.ui.js` DOM/render/pagination/permission helpers; `app.tables.js` declarative tables; `app.workflow.js` status transitions; `app.upload.js` drag/drop uploads. Page modules fetch from the API, render HTML strings, and call `lucide.createIcons()`. Routes use the central `FrontendRoutes` map (`url_for()` in PHP, `APP.route()` in JS); physical filenames are never public. The admin/portal shells set `APP.currentUser`, apply permission gating, and dispatch `app:ready`.

11 Design System and Theming

Brand palette `-brand-50--brand-900` anchored on `-brand-600 #8B0A72`; neutral `-ink` scale; semantic success/warning/danger/info colours; radii and shadows; Inter body + Playfair Display headings. CSS files: `core.css` (tokens/reset), `components.css` (buttons, cards, tables, badges, modals, forms, timeline, totals, avatars, skeletons, loaders), `layout.css` (admin shell), `utilities.css`, `site.css` (public + portal), `help.css`. Tailwind CDN extends the brand palette and fonts. Light theme only, desktop-first, responsive.

12 CDN and Third-Party Libraries

Tailwind CSS (CDN utility styling), Flowbite 2.5.1, Lucide icons, ApexCharts (reports), Flatpickr (date pickers, lazy), Swiper 11 (carousels, lazy), Google Fonts (Inter, Playfair Display). Registered in `AssetManager::LIBRARIES`, opted into per page via `$extraLibraries`.

13 Database Overview

Name `wedding_planner`, collation `utf8mb4`, 29 tables, InnoDB. Money in `DECIMAL(12,2)` (integer cents in PHP via `Money`). Reference numbers `PREFIX-YYYY-NNN` via `Reference::next()`. `created_at/updated_at` on mutable records. Status enums as short strings surfaced as badges.

14 Database Tables Reference

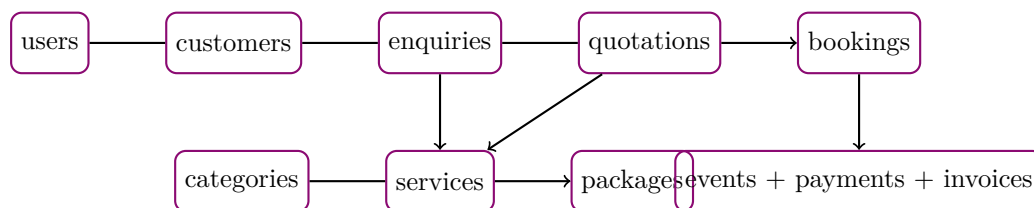
All 29 tables (full DDL in `database/schema.sql`):

#	Table	Purpose
1	<code>roles</code>	Role definitions (admin, manager, staff, customer)
2	<code>users</code>	Login accounts (all roles)
3	<code>customers</code>	Customer profiles (optionally linked to a user)
4	<code>service_categories</code>	Catalogue categories
5	<code>services</code>	Bookable services with pricing/duration/image
6	<code>service_images</code>	Per-service gallery images (primary + ordering)
7	<code>packages</code>	Bundled packages with discount rules + cover image
8	<code>package_services</code>	Package-to-service links with quantities
9	<code>offers</code>	Promotional offers (percent/fixed, dated)
10	<code>offer_services</code>	Offer-to-service applicability
11	<code>offer_packages</code>	Offer-to-package applicability
12	<code>enquiries</code>	Website/portal enquiries with reference numbers
13	<code>enquiry_services</code>	Enquiry line items (service or package)
14	<code>quotations</code>	Quotations with totals and validity
15	<code>quotation_items</code>	Quotation line items (snapshotted)
16	<code>bookings</code>	Confirmed bookings
17	<code>booking_services</code>	Booking line items (snapshotted)
18	<code>events</code>	Scheduled events per booking
19	<code>staff</code>	Staff members (optional login link)
20	<code>staff_assignments</code>	Event-to-staff assignments
21	<code>payments</code>	Recorded payments (recorded/reversed)

#	Table	Purpose
22	invoices	Invoices issued against bookings
23	reviews	Customer reviews with moderation state
24	leads	Sales leads and their status
25	lead_followups	Lead follow-up entries
26	notifications	In-app notifications
27	activity_logs	Audit trail
28	settings	Key/value business settings
29	login_attempts	Login success/failure tracking for rate limiting

15 Entity Relationships

users 1:1 customers; customers 1:N enquiries 1:N enquiry_services; enquiries 1:0..1 quotations 1:N quotation_items; quotations 1:0..1 bookings 1:N booking_services; bookings 1:N events 1:N staff_assignments N:1 staff; bookings 1:N payments/invoices/reviews; service_categories 1:N services 1:N service_images; packages N:M services via package_services offers N:M services/packages; roles 1:N users; leads 1:N lead_followups.



16 Installation (XAMPP)

Install XAMPP (Apache + MySQL + PHP 8.2+). Place the project at `C:\xampp\htdocs\Wedding Planner`. Start Apache and MySQL. Create database `wedding_planner` (utf8mb4), import `database/schema.sql`, run `database/seed.php` with the XAMPP PHP binary. Verify `config/config.php`. Open `http://localhost/Wedding%20Planner/` (public site) and `/manage/login` (admin). Customer login lives on the website and opens `/account`.

17 Configuration Reference

`config/config.php` returns one array: `app_name` (VivaahFlow), `timezone` (Asia/Kolkata), `debug` (false), `http.host/scheme` (from request), `database host/port/name/user/pass/charset` (127.0.0.1/3306/wedding_planner/root/empty/utf8mb4), `session.name` (vivaahflow_sess), `session.lifetime` (7200), `session.same_site` (Lax), `session.secure` (false), `security.login_max_attempts` (5), `security.login_lock_minutes` (15), `security.upload_max_bytes` (3145728, 3 MB), `security.allowed` (jpg, jpeg, png, webp), `pagination.default_per_page` (10), `paths.uploads/logs`, `currency` (INR, 2 decimals), `api_base` (/api/v1).

18 Database Setup and Seeding

Create the schema, import `database/schema.sql`, then run the seeder, which prints the demo credentials. The seeder loads roles, users, catalogue, CRM, finance, and reporting demo data. Reseeding a non-empty database is guarded — reset via a fresh schema import first.

19 Default Credentials

Demo accounts (change all passwords before any real deployment): admin `admin@example.com` / Admin@123; manager `manager@example.com` / Manager@123; staff `staff@example.com` / Staff@123; customer `customer@example.com` / Customer@123.

20 Running with Apache

Root `.htaccess` routes `/`, `/services`, `/packages`, `/account`, `/manage` to the frontend routers with clean URLs; the API is `/api/v1/index.php/<route>` via `PATH_INFO` (`AcceptPathInfo` On). Public `/assets/...` served from `frontend/assets/`; `/uploads/` publicly readable. Directory listing disabled. `backend/`, `config/`, `database/`, `storage/`, and direct `frontend/` access return 403. Sensitive files (`.env`, `.sql`, `.log`, `README.md`, `.bak`, `.tmp`, dotfiles) denied. Missing asset files fall through to the customer router (which returns the homepage HTML with status 200) — so every referenced asset file must physically exist.

21 Running with the PHP Built-in Server

`php -S 127.0.0.1:8100 -t "C:\xampp\htdocs\Wedding Planner"`. Browse `http://127.0.0.1:8100/`, `/services`, `/account`, `/manage/login` (no `/Wedding Planner` prefix; `.htaccess` not used; routers accept the internal `?route=` parameter).

22 Page and URL Map — Public Website

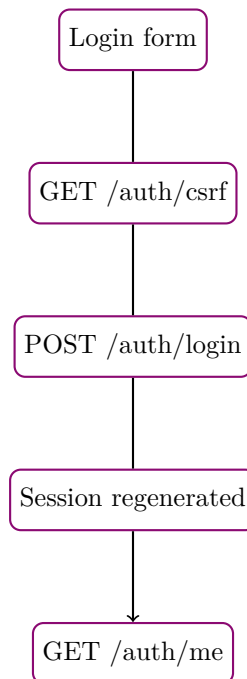
`/` (`home.php`), `/services` + `/services/{slug}`, `/packages` + `/packages/{slug}`, `/gallery`, `/offers`, `/reviews`, `/contact`, `/login`, `/register` — all under `frontend/customer/` with shared header/footer partials and `customer/site.js`.

23 Page and URL Map — Portal and Admin

Portal (`frontend/portal/`): `/account`, `/account/enquiries`, `/account/quotations + /{id}`, `/account/bookings + /{id}`, `/account/payments`, `/account/invoices + /{id}`, `/account/reviews`, `/account/profile`. Admin (`frontend/admin/`): `/manage` dashboard, customers + `/{id}`, enquiries, leads, categories, services, packages, offers, quotations + `/{id}`, bookings + `/{id}`, events, staff, payments, invoices + `/{id}`, reviews, reports, notifications, users, settings, activity, profile, help, login.

24 Authentication Flow

POST /auth/login validates email + password, checks rate-limit state, verifies the hash; on success the session regenerates and stores user id/role/name/email. `AuthService::user()` loads the user with role plus linked customer profile, cached per request. GET /auth/me returns the shaped user; POST /auth/logout clears the session. The admin footer sets `APP.currentUser` and applies permissions; the portal is guarded by `EnsureCustomer` server-side and `portal_guard()` in partials. Login requires a prior GET /auth/csrf token for the session.



25 Session Management

Cookie `vivaahflow_sess`, lifetime 7200 s, `SameSite=Lax`, `secure` off by default (enable behind HTTPS). Sessions start in `backend/bootstrap.php`; id regenerates on login; `Session::csrfToken()` renders into `<meta name="csrf-token">`.

26 CSRF Protection

`CsrfGuard` rejects mutating requests without a valid token. The token ships in `<meta>` and `APP.csrfToken`; `app.api.js` attaches `X-CSRF-Token` automatically; a fresh token is available at GET /auth/csrf.

27 Security and Rate Limiting

Bcrypt hashing; attempts logged in `login_attempts`; 5 failures within 15 minutes temporarily block login (429). All queries use prepared statements. Uploads validated by size (3 MB) and extension, stored under generated names. UI output escaped (`UI.escape/e()`). Role checks run server-side; the UI additionally hides unauthorised controls. Errors log to `storage/logs/php-error.log`, never exposed in API responses.

28 Roles Overview

admin: full access. **manager:** everything except destructive/administrative areas (settings, users, roles, activity, selected deletes/reversals). **staff:** operational subset (dashboard, customers read, bookings read/update, events, staff read, assignments, follow-ups, review moderation, notifications). **customer:** portal-only own data.

29 Permissions Matrix

Area	admin	manager	staff	customer
Dashboard	yes	yes	yes	—
Customers	full	read/create/update	read	self
Enquiries	full	read/update/convert	—	own (portal)
Leads & follow-ups	full	full	follow-ups	—
Catalogue	full	full	—	public read
Quotations	full	full	—	own accept/reject
Bookings	full	full	read/update	own
Events & assignments	full	full	scoped	—
Staff	full	full	read	—
Payments	full	create (no reverse)	—	own history
Invoices	full	read/update	—	own
Reviews	full	moderate (no delete)	moderate	submit own
Reports	yes	yes	—	—
Notifications	yes	yes	read/update	—
Settings	full	—	—	—
Users & roles	full	—	—	—
Activity log	yes	—	—	—

30 Full Permission Reference

`dashboard.view`; `customers.read/create/update/delete`; `services.read/create/update/delete`; `categories.read/create/update/delete`; `packages.read/create/update/delete`; `enquiries.read/update/delete`; `quotations.read/create/update/status`; `bookings.read/create/update/status`; `events.read/create/update/delete`; `staff.read/create/update/delete`; `assignments.manage`; `payments.read/create/reverse`; `invoices.read/create/update/delete`; `reviews.read/moderate/delete`; `offers.read/create/update/delete`; `leads.read/create/update/delete`; `followups.manage`; `reports.read`; `notifications.read/update`; `settings.read/update`; `users.read/create/update/delete`; `roles.read`; `activity.read`. Managers receive everything except `customers.delete`, `invoices.delete`, `invoices.create`, `reviews.delete`, `payments.reverse`, all `settings.*`, `users.*`, `roles.*`, and `activity.read`.

31 Admin Panel Overview

Composed from `admin/partials/`: `bootstrap.php` (backend load, access gate), `sidebar.php` (grouped nav, active highlight, brand mark), `header.php` (title, notifications, quick actions), `footer.php` (shared scripts, `APP.currentUser`, permission application, `app:ready`, notification polling). Each page sets `$pageTitle`, `$pageDesc`, `$activeNav`, `$pageScript`, optionally `$extraCss`, `$extraLibraries`, `$areaScripts`.

32 Dashboard Module

`admin/dashboard.php` + `js/admin/dashboard.js` on `GET /reports/dashboard` and `GET /dashboard`: KPI cards (customers, enquiries, bookings, revenue, pending payments, completed services, follow-ups, pending quotations, active offers, staff), revenue trend, booking-status distribution, upcoming events, pending follow-ups, shortcuts.

33 Customers Module

List (`customers.php`) and detail (`customer.php`): searchable, status-filtered, paginated; create/edit; active/inactive toggle; permission-gated delete. Detail aggregates enquiries, quotations, bookings, payments, invoices, reviews. API: `GET/POST /customers`, `GET/PUT /customers/{id}`, `PATCH /customers/{id}/status`, `DELETE /customers/{id}`.

34 Enquiries and Leads Modules

Enquiries: statuses `new`, `contacted`, `quotation_pending`, `quotation_sent`, `converted`, `closed`; detail with requested items; convert to quotation/lead. Leads: pipeline with source, status (`new`, `contacted`, `follow_up`, `converted`, `lost`), assignment, follow-up scheduling and history, upcoming endpoint.

35 Catalogue Modules

Categories (name, slug, description, status, ordering). Services: CRUD with category, `starting_price`, duration, featured flag, status, ordering, cover image; gallery sub-resource (list/add/update, set primary, delete). Packages: bundled services with quantities, percent/fixed discounts; `CalculationEngine` base/discount/total; discount row hidden when zero. Offers: time-bound percent/fixed, applicable to all/services/packages; public site shows active non-expired only.

36 Quotations Module

Generated from enquiries; editable line items; discounts. Statuses `draft`, `sent`, `accepted`, `rejected`, `expired`; direct `draft-to-terminal` transitions rejected with 409. Customers accept/reject via portal; expired quotations cannot be accepted; converted/rejected quotations cannot spawn bookings.

37 Bookings Module

Created from accepted quotations with snapshotted line items. Statuses `pending`, `confirmed`, `scheduled`, `in_progress`, `completed`, `cancelled`. List offers schedule/payment modals with remaining-balance awareness; detail combines summary, services, events with assignments, payment history. `payment_status` (unpaid/partial/paid) derived server-side. Deletion permission-gated.

38 Events and Staff Assignments Module

Events belong to a booking (date/time, venue, title, status `scheduled`, `in_progress`, `completed`, `cancelled`). Day Schedule panel via GET `/events/schedule?date=&status=`. Assign staff, mark complete, remove. Full sub-resource API under `/events`.

39 Staff Module

Directory with designation/specialty and optional linked login (requires `create_login`, valid email, password; otherwise skipped). Status toggle and CRUD; active-staff endpoint for assignment pickers.

40 Payments and Invoices Modules

Payments recorded against bookings; client validates against remaining balance, server authoritative; over-payment and negative balances prevented; reversal (`recorded` to `reversed`) permission-gated. Invoices generated from bookings with issue/due dates; lifecycle `draft`, `issued`, `partial`, `paid`, `overdue`, `cancelled` (admin status endpoint accepts `draft`, `issued`, `cancelled`); paid/partial/overdue derived from payments; detail print-optimised.

41 Reviews Module

Moderation queue with stars and visibility control; statuses `pending`, `approved`, `rejected`; business reply attachable. Public site shows approved, visible reviews only.

42 Reports and Analytics Module

ApexCharts dashboard: KPI snapshot (`/reports/dashboard`, `/reports/summary?range=3m|6m|12m`); revenue area, booking-status donut, quotation/lead funnels, service performance bars; date-ranged revenue (`/reports/revenue?from=&to=`) and bookings (`/reports/bookings?from=&to=`) reports; top customers; recent activity.

43 Notifications, Settings, Users, Activity, Profile Modules

Notifications: `{items, unread}`, all/unread filter, mark-read/mark-all-read, header indicator with polling. Settings: company profile, tagline, contact, currency symbol, footer, `registration_open`; logo upload via multipart POST `/settings/logo` (field `logo`). Users: role assignment, status toggle, deletion with self-protection. Activity: searchable audit trail (actor, action, entity, IP, timestamp). Profile: current-user summary, contact update, password change with verification.

44 Public Website

Home: white premium hero, trust/stat strip, categories, featured services, one featured package, gallery showcase, one active offer, testimonials; empty sections hidden automatically. Services: search + category chips; detail with gallery, reviews, similar services, sticky enquiry card. Packages: catalogue + detail with conditional discount row. Gallery renders uploaded media with a

friendly empty state. Offers: active non-expired only. Reviews: average, distribution bars, list. Contact: enquiry form with service/package selection, signed-in prefill, reference-number confirmation. Auth: customer login/registration honouring `registration_open`. Shared header (sticky nav, session-aware actions), footer (branding, links, contact), and `customer/site.js`.

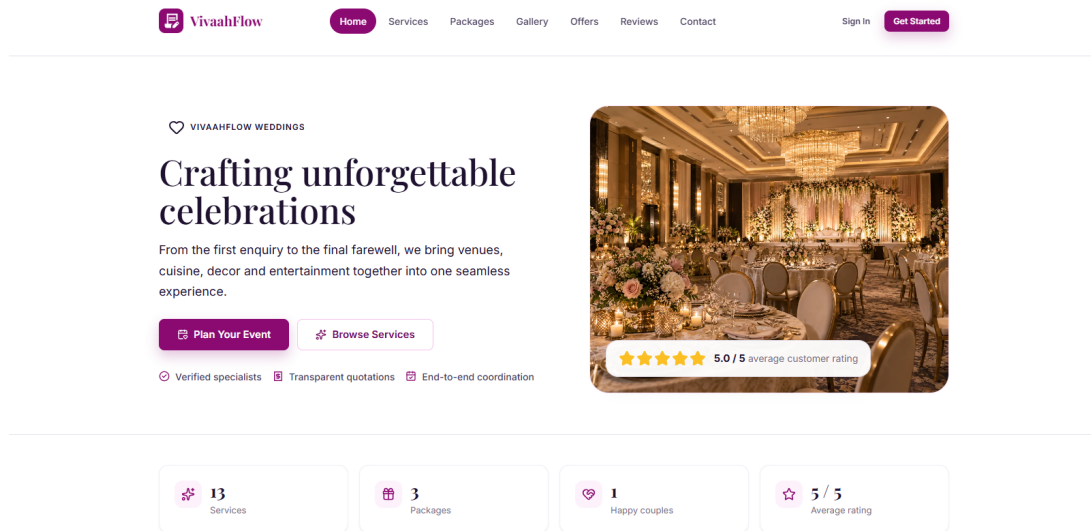


Figure 1: Public homepage (live data, September 2026).

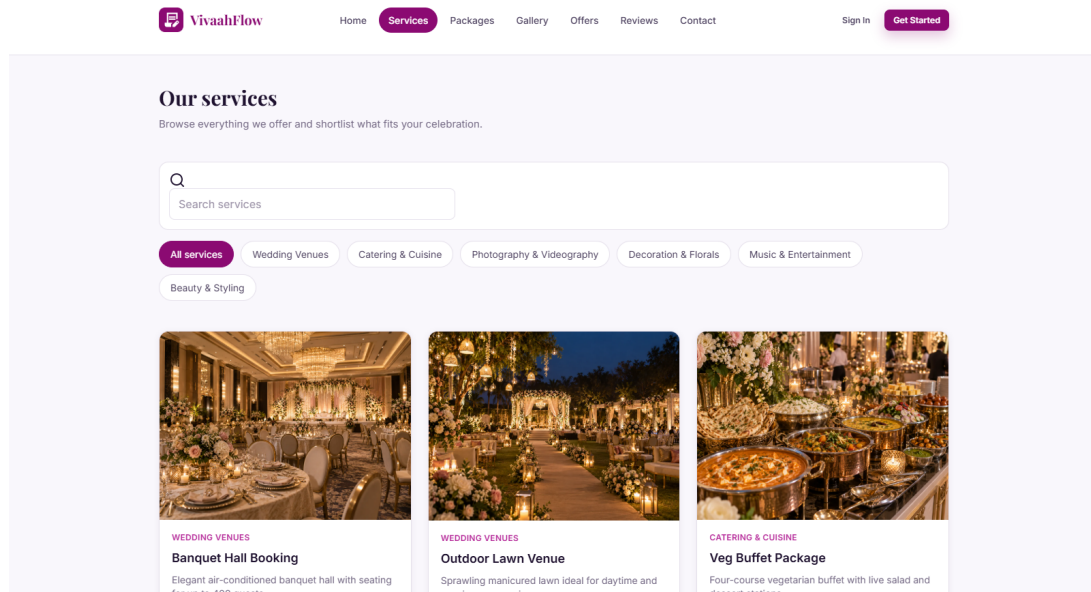


Figure 2: Service catalogue with search and category filtering.

45 Customer Portal

Gated under `/account` (`portal_guard()` + `EnsureCustomer`); anonymous visitors redirect to sign-in with `next`. Dashboard (derived stats, upcoming events, recent enquiries, quick actions); enquiries (read-only history; new via contact form); quotations (list + detail; accept non-expired

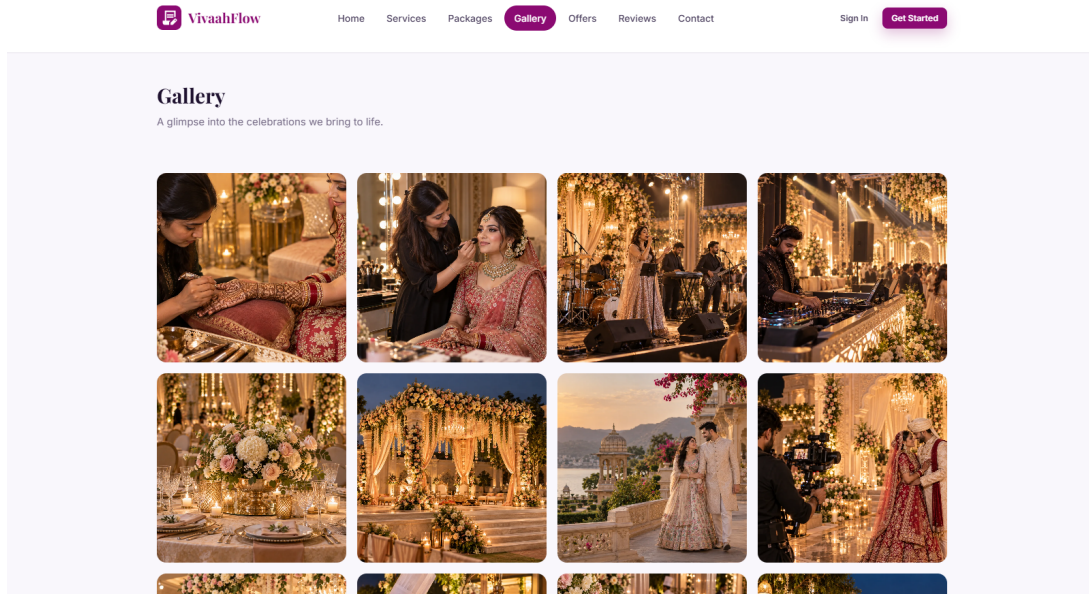
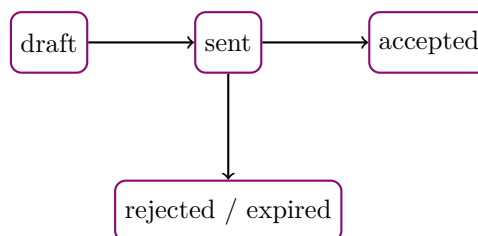


Figure 3: Gallery showcase rendered from uploaded media.

sent, decline with optional reason); bookings (list + detail with payment summary, services, schedule, staff); payments (history + running total); invoices (list + printable detail); reviews (submit for completed bookings; moderation status; replies); profile (contact/celebration details, password change). API: /portal/profile, /portal/enquiries, /portal/quotations..., /portal/bookings..., /portal/payments, /portal/invoices..., /portal/reviews....

46 Booking and Status Lifecycles

Enquiry: new to contacted to quotation_pending to quotation_sent to converted (or closed). Quotation: draft to sent to accepted / sent to rejected / sent to expired. Booking: pending to confirmed to scheduled to in_progress to completed (or cancelled). Event: scheduled to in_progress to completed (or cancelled). Payment: recorded to reversed. Invoice: draft to issued to partial to paid (or overdue/cancelled). Review: pending to approved/rejected. Lead: new to contacted to follow_up to converted (or lost). Catalogue entities: active/inactive. The UI transition helpers (Workflow.quotationNext/bookingNext/eventNext) propose next states; the server validates every transition.



47 Calculation Engine and Money Handling

Money as integer cents via App\Helpers\Money (toCents, fromCents, format); storage DECIMAL(12,2). CalculationEngine::discountAmount() resolves percent (base x value / 100) or fixed discounts; PackageService::hydrate() enriches base/discount/total; zero discounts produce no

label and hide conditional rows. Totals computed server-side with `raw + *_formatted` fields. Booking `paid_amount/remaining_amount` arrive pre-formatted. Client: `APP.money(cents)`, `Workflow.amount(rupees)`, `Site.money(rupees)` (Indian grouping). Payment status derived, never stored.

48 File Uploads and Media Handling

Validated by size (3 MB) and extension (jpg/jpeg/png/webp), stored under generated names. ImageEngine stores/replaces and reference-counts files across `services.image`, `service_images.image_path`, and `packages.cover_image`. `upload_url()` builds public URLs. Customer JS never renders blank boxes: every image site falls back to a Lucide icon (`Site.image`, service/package cards, gallery figures), and empty collections render friendly empty states. The admin services table shows an `image-thumb` or icon; the gallery modal shows a “No gallery images yet” state. The brand mark is the single canonical file `assets/images/brand/logo.png` (192×192 PNG — the only file in `images/brand/`), referenced by all headers, the admin sidebar, the admin login, the home hero mark, and the page favicon; `logo.svg` does not exist and is referenced nowhere. Additional favicon assets (“`favicon.ico`, `apple-touch-icon.png`, `PWA icon-192/512.png`) live in `assets/icons/`.

49 Media Integration Pass (September 2026)

Twenty-three raw PNG photographs (56.2 MB total) were analysed, renamed descriptively, and converted to 26 WebP files (5.4 MB total): 23 service images plus 3 package covers. `media_integration.sql` applied: 13 `services.image` values set, 23 `service_images` rows inserted (primary + supporting), 3 `packages.cover_image` values set, and one leftover package residue (id 4) with its links deleted. Verified state: `services` with image 13; `service_images` 23; `packages` 3, all with covers; zero gallery rows orphaned from services without images. Public API: `/public/services` returns 13 services each with `image_url`; `/public/gallery` returns 23 items; `/public/packages` returns 3 packages all with `cover_image_url`. Example: `banquet-hall-booking` serves `uploads/services/banquet-hall-grand-01.webp`. Full mapping in `docs/image-inventory.json` and `docs/image-inventory.md`.

50 REST API Reference

Single entry point `/api/v1/index.php` with `PATH_INFO` routing and `?route=` fallback: 144 routes in 24 groups (each `{id}/{slug}` a path parameter; panel routes require session role; portal routes require customer role):

Method + path	Handler
GET <code>/health</code>	HealthController@index (st
POST <code>/auth/login register logout</code>	AuthController
GET <code>/auth/csrf me</code>	AuthController
POST <code>/auth/change-password</code> , PUT <code>/auth/profile</code>	AuthController
GET <code>/public/bootstrap home services packages gallery reviews offers</code>	PublicController
GET <code>/public/services/{slug}</code> , GET <code>/public/packages/{slug}</code>	PublicController
POST <code>/public/enquiries</code>	PublicController@submitEn
GET <code>/dashboard</code>	DashboardController@inde

Method + path	Handler
GET/POST /customers, GET/PUT /customers/{id}	CustomerController (panel)
PATCH /customers/{id}/status, DELETE /customers/{id}	CustomerController
GET/POST /categories, PUT /categories/{id}, DELETE	CategoryController
GET/POST /services, GET/POST /services/{id}	ServiceController
PATCH /services/{id}/status, DELETE /services/{id}	ServiceController
GET/POST /services/{id}/gallery + update/primary/delete	ServiceController
GET/POST /packages, GET/POST /packages/{id}	PackageController
PATCH /packages/{id}/status, DELETE /packages/{id}	PackageController
GET /enquiries, GET /enquiries/{id}	EnquiryController
PATCH /enquiries/{id}/status, POST .../quotation lead	EnquiryController
GET /quotations, GET POST /quotations/{id}	QuotationController
PATCH .../status, POST .../accept reject	QuotationController
GET/POST /bookings, GET POST /bookings/{id}	BookingController
PATCH .../status, POST .../schedule, DELETE	BookingController
GET /events, GET /events/schedule, POST /events	EventController
GET POST /events/{id}, PATCH, DELETE	EventController
POST /events/{id}/assignments, assignment complete/delete	EventController
GET /staff, GET /staff/active, POST /staff	StaffController
GET PUT /staff/{id}, PATCH, DELETE	StaffController
GET POST /payments, GET /payments/{id}	PaymentController
POST /payments/{id}/reverse	PaymentController
GET POST /invoices, GET /invoices/{id}	InvoiceController
PATCH /invoices/{id}/status, DELETE	InvoiceController
GET /reviews, GET /reviews/{id}	ReviewController
PATCH /reviews/{id}, DELETE /reviews/{id}	ReviewController
GET POST /offers, GET PUT /offers/{id}	OfferController
GET /offers/active, PATCH, DELETE	OfferController
GET POST /leads, GET PUT /leads/{id}	LeadController
GET /leads/upcoming, PATCH, POST assign followups, DELETE	LeadController
GET /reports/dashboard summary revenue bookings	ReportController
GET /notifications, POST read-all {id}/read	NotificationController
GET /settings, PUT /settings, POST /settings/logo	SettingsController
GET /users, GET /roles, POST /users	UserController
GET PUT /users/{id}, PATCH, DELETE	UserController
GET /activity	ActivityController@index
GET PUT /portal/profile, GET POST /portal/enquiries	PortalController
GET /portal/quotations + /{id} + accept/reject	PortalController
GET /portal/bookings + /{id}, GET /portal/payments	PortalController
GET /portal/invoices + /{id}	PortalController
GET /portal/reviews eligible, POST /portal/reviews	PortalController

Error contract: 401 unauthenticated, 403 forbidden, 404 not found, 409 illegal transition, 422 validation (with errors map), 429 login lockout. Verified live: GET /health 200 (status ok, database ok); GET /auth/csrf success; unauthenticated GET /customers 401; unknown route 404; all 20 page/asset HEAD checks 200 except the former path-style health probe (resolved: GET works in all three URL forms).

51 Validation and QA Evidence

Final pass, September 2026 (XAMPP Apache :80, MySQL 3306, PHP 8.2.12, Node 24, Chrome via playwright-core):

Check	Result
<code>php -l</code> over project	156/156 OK
<code>node -check</code> over project JS	58/58 OK
Image audit pages (all HTTP 200)	9/9, 0 broken, 0 blank, 0 empty-src
— home / services / gallery imgs	15 / 14 / 24, brand logo present
— service-detail / packages / package-detail	4 / 4 / 2
— offers / reviews / contact	1 / 1 / 1
Admin logo (<code>logo.png</code> 192x192 source, rendered 40px)	pass
Viewport sweep (5 viewports x 6 routes)	30/30, 0 overflow, 0 errors
Public pages + assets + auth pages (HEAD)	20/20 status 200
API health (3 URL forms, GET)	3/3 status 200 JSON
API media: services 13/13 with <code>image_url</code>	pass
API media: gallery 23, packages 3/3 with covers	pass
DB: services w/ image 13, <code>service_images</code> 23	pass
DB: packages 3/3 with cover, orphan gallery rows 0	pass
Auth journeys (prior baseline)	20/20
API regression incl. 409/422/403/401/404	8/8
Interactions	6/6
<code>php-error.log</code>	no PHP errors logged (directory empty)
Temp test records residue	0

52 Real-Browser Repair Pass (September 2026)

Sixteen headed (visible) Chrome sessions exercised every critical workflow end to end, found five genuine runtime defects, and retested each fix to PASS with database proof (full narrative in `QA_REAL_BROWSER_REPORT.md`, fix details in `MASTER_FIX_REPORT.md`): (1) a permission race that permanently hid row-action buttons when a table rendered before `/auth/me` — fixed with two-way `applyPermissions` plus an `app:ready` re-apply in `Tables.create` (forced-race retest 30/30 buttons, 3 runs); (2) package create/update with services always returning 422 — `PackageController` now normalizes the JSON-string `service_ids` before validation; (3) settings logo upload never sending anything (`API.upload` argument order) — now POST 200 with DB and file verified; (4) checkbox selections wiped by slow reference-list loads — checked state preserved; (5) horizontal overflow at 768 px (footer grid blowout, +32) and contact at 1024 px (+89) — grid/detail-row shrink-and-wrap guards, responsive suite **35/35 PASS**. Verified lifecycles: profile edit (PUT 200, no reload, hard-refresh persistence), services / packages / offers / reviews / gallery / customers CRUD with zero DB residue, logout, registration (201), guest enquiry (201, `ENQ-2026-008`), authorization matrix (403/401/redirects), 20 admin pages with 0 console errors and 0 failed requests, and mobile drawer navigation. A second full role/CRUD visible pass (admin/manager/staff/customer + QA user lifecycle) found four more defects, all fixed and retested: password-change and admin-profile 401s (routes lacked user resolution), enquiry-to-lead 500 (wrong data keys), unlinked logged-in enquiries, and stale names after re-name. Responsive suite now 45/45 across 9 viewports; sidebar 21/21 verified.

53 Troubleshooting, Error Handling and Deployment

Port conflicts (80/3306): stop competing software or reboot XAMPP. Missing-asset-returns-homepage: create the physical file under `frontend/assets/` (the canonical brand mark is `images/brand/logo.png`). API 401/403: check role and session; 422: inspect `errors` map; 409: follow the lifecycle in Section 46; 429: wait out the 15-minute lockout. Blank image boxes: never CSS-hide; fix the data/URL and keep the icon fallback. Before deployment: change all demo passwords, set `debug` false, enable `secure` cookies behind HTTPS, restrict exposed directories, back up `wedding_planner` and `uploads/`, and re-run this chapter's QA table.

54 Release Checklist

Media integrated (13 services, 23 gallery rows, 3 covers) with zero broken/blank images; logo present in all shells; lint clean (156 + 58); browser QA green (9 + 30 checks); real-browser repair pass complete (5 runtime defects fixed and retested, 35/35 responsive, CRUD lifecycles with DB proof — see `QA_REAL_BROWSER_REPORT.md` and `MASTER_FIX_REPORT.md`); API + DB counts verified; error log clean; temp records removed; documentation compiled (PDF + editable DOCX); README rewritten to match this release.